Scripting for Cybersecurity Project

PROCESS & IP ADDRESS ANALYSIS TOOL

B00171248 LEXIE WALSH, B00171841 ADAM LAMONT

School of Informatics & Cybersecurity - Assessment Submission Coversheet

Student Information

Student Number:	B00171248, B0017184
Student Name:	Lexie Walsh, Adam Lamont

Module Information

Module Title:	CrossList-Scripting for Security
Module Code:	COMP-H2702

Assessment Information

Assessment Title:	Automating a Cybersecurity Workflow

Declaration

✓ I confirm that I have read and understood the University's Academic Integrity Policy.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/academic-quality-assurance-and-enhancement/academic-integrity/

✓ I have completed the TU Dublin AI Driving License theory test.

https://www.tudublin.ie/explore/about-the-university/academic-affairs/lta/lta-resources/artificial-intelligence-driving-license/

✓ I declare that this assessment submission is my own work.

✓ I confirm that I have read and understood the AI traffic light system used as part of this assessment brief and I have adhered to the level of AI usage allowed for this assessment.

Student signature:	Lexie Walsh, Adam Lamont
Date:	12.12.2025

Table of Contents

Introduction 1
Tool Design & Implementation 2
PowerShell Component 2
Python Component 2
Cybersecurity Context 3
Tool Demonstration 4
Reflection 6
Division of Work 6
Reflection – Lexie Walsh 7
Reflection – Adam Lamont 8
References 9
Adam Lamont 9
Lexie Walsh 9

Introduction

For this project, we created a cybersecurity scripting tool that uses both PowerShell and Python. It uses PowerShell to retrieve active processes (any with > 5% CPU usage) and their associated network connections and data such as their hash and file location.

This is placed into a PowerShell custom object, which creates a JSON file which is fed into a Python script which queries AbuseIPDB (for IP addresses) and VirusTotal (hashes of executables, and produces a report in markdown format, along with using matplotlib to create a bar chart of the most commonly connected IP addresses.

Tool Design & Implementation

PowerShell Component

PowerShell is used both for an orchestrator script to run all components, and an analysis script to perform enumeration of processes and their network connections. Everything in the analysis script is contained within separate functions, which combines the results of these functions into a single custom object. This custom object is then exported as a JSON file.

The Orchestration script also captures the output of the Python analysis component as a variable and outputs it to a .md file, so all output is done through PowerShell.

Python Component

The Python script takes the information gained from the analysis script, filters through all the information given and creates a new, JSON-formatted dictionary for each unique program running. For each program, its name, all Process IDs affiliated with it, the path, its hash, its remote IP addresses and some other relevant information were captured.

To avoid duplicate values, sets were used when adding information, which later changed to lists if the set contained more than one item, or reverted to its original format. If there were multiple programs with the same name, but different paths, a new program would be added to the dictionary to avoid separate programs being put into the same dictionary.

The hashes and IP addresses were later checked using Virustotal and AbuseIPDB to check if they are known as malicious hashes (for Programs) and if any of the remote IP addresses are known to be suspicious and could pose a threat.

The way the dictionary was designed, it could be written to a JSON file if someone wanted to.

There is then a function at the end of the Python script which collects the data that was extracted from the outputs from the APIs. This produces a basic report in markdown format. This report contains each process, data about the processes such as its hash and file location and whether VirusTotal detected it as malicious, and the IP addresses associated with the remote network connections that the process makes, and whether they are malicious or not according to AbuseIPDB.

An aggregate of the frequency of certain IP addresses being connected to is also printed into the report, along with using matplotlib (example of external library) to produce a graph.

Cybersecurity Context

This tool can be used by an administrator to detect potential malicious processes, as if a process is connecting to a malicious IP address and using a moderate amount of system resources, there is a high chance that it could be malicious (e.g. phoning home to a command and control server), and the outputs that this tool provides could be used to alert the administrator to this.

It could also simply show a system or network administrator, the most frequently connected to IP addresses by in-use processes, which could provide them with data on what employees could be connecting to or what IP addresses the organization connects to.

The analysis.ps1 script also collects additional data which isn't used by the Python component, such as creation dates of the processes, so the raw JSON has some additional use for a cybersecurity professional. It could also be run independently of the Python component.

Tool Demonstration

PowerShell Version used for testing: **5.1.26100.6899**

Python Version used for testing: **3.14.0**

To run this code, extract all the files into a single directory, run pip install –r requirements.txt (for requests and matplotlib libraries, which are not in the standard library) and run .\Orchestrator.ps1. The code was only tested on Windows systems. While it should run on Linux, the output of the cmdlets are likely to be different and it may cause issues.

The code should run, and all outputs are stored in the directory that the user is currently in.

```
PS C:\Users\al\Documents\College Work\Year 2 Semester 1\Scripting for Security\Labs\scriptingForCybersecurityProject> .\orchestrator.ps1
Running scripts..
Running Powershell script..
Running python script..
Successfully saved result to finalOutput.md
```

Sample of output file (finalOutput.md):

```
# Processes and Connections Report:
## Process Analysis

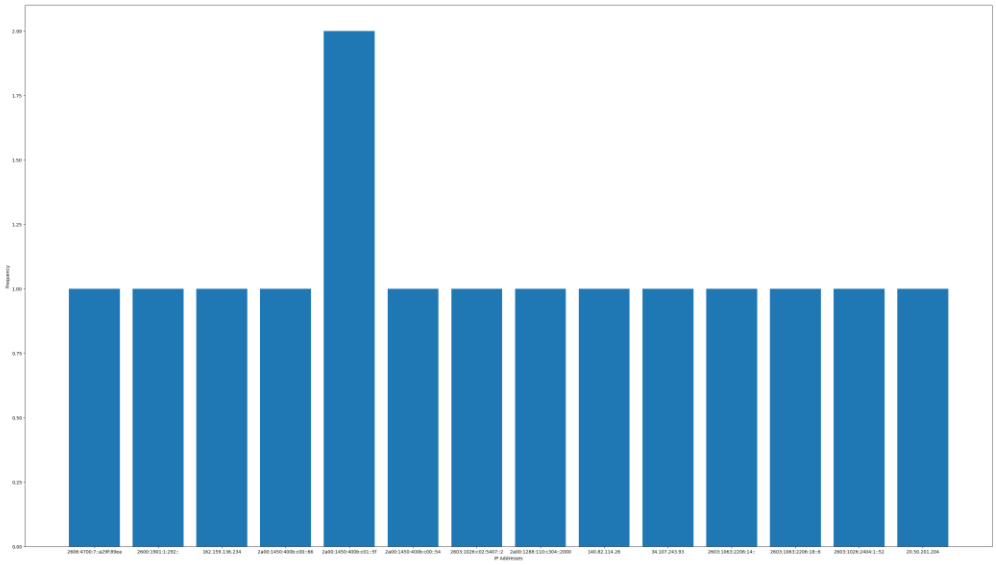
### Process: ctfmon

Path: None
PID: 12752
Hash: None
#### VirusTotal Analysis:
VT Score: None
AV Results: (Checked by None engines, None flagged it as malicious.)
---
### Process: explorer

Path: C:\WINDOWS\Explorer.EXE
PID: 8592
Hash: 2066651BCBA4BA23B79B45C813994FE7E7B71A29E0F50915A9D0681BCE4D4A50
#### VirusTotal Analysis:
VT Score: 0/72
AV Results: (Checked by 72 engines, 0 flagged it as malicious.)
---
### Process: firefox

Path: C:\Program Files\Mozilla Firefox\firefox.exe
PID: [16160, 13732, 14116, 15736, 11240, 11948, 5100, 10796, 13104, 15412, 6776, 15448, 13404]
Hash: 5263F4DB29950465D0A74487A9F5BDE55ACF034DED43CB4CC4673A1084EAECA0
#### VirusTotal Analysis:
VT Score: 0/71
AV Results: (Checked by 71 engines, 0 flagged it as malicious.)
---
#### Remote Network Connections:
IP: 34.107.243.93
Country: US
Usage: Content Delivery Network
AbuseIPDB Confidence Score: 70
---
```

Sample of data visualization:



Reflection

Division of Work

Lexie Walsh:

- Python JSON reading and picking out useful information – I did this because it will be easier management of information and avoiding duplication (e.g. IP addresses, hashes)
- API requests to VirusTotal for hashes and AbuseIPDB for IP addresses – This was done to check if a hash/IP address is known for being malicious/suspicious and adding it to the variable with the rest of the information.
- Report

Adam Lamont:

- PowerShell orchestrator and analysis scripts – Chosen as I prefer PowerShell over Python.
- Python data aggregation and visualization components – Chosen because I have used data visualization tools in Python for previous projects.
- Readme for code.
- Report

Reflection – Lexie Walsh

While working with the JSON file, I have gained extensive knowledge on how to search through and make JSON-like dictionaries in Python.

A struggle I faced was after gaining the unique information of each process through sets, converting them to lists if the set had more than one value in it, and removing the set type from the rest, then converting them back to their original form (string or integer). This was eventually solved using the isinstance(), len(), next() and iter() functions:

```
for key, value in results[name].items():
    if isinstance(value, set):
        if len(value) > 1:
            results[name][key] = list(value)
        else:
            results[name][key] = next(iter(value))
```

With this done, now every set with more than 1 item was converted to a list, and the rest was converted back to its original form.Another challenge was attempting to convert a list of IP addresses to be keys in their own dictionaries, so that I can put results from AbuseIPDB in it. While attempting to do this, I have learned that it is possible to put loops into dictionaries, to create new dictionaries with their initial values:

```
if isinstance(ip, list):
    results[name]["Ip"] = {i: {} for i in ip}
```

With this, I was then able to add information to each unique IP address.

Reflection – Adam Lamont

I think that this project enhanced my knowledge of using PowerShell to enumerate data about a Windows system into a custom object and then convert it to JSON to insert it into a Python script.

A notable challenge that I faced and could not find a solution to was that I was unable to get Python to work with the JSON being piped directly into the Python code, (i.e. .*analysis.ps1* | *python "lookup.py"* and using stdin to gather this input). The JSON was loaded into Python as an object (if you printed the input from sys.argv/stdin it printed the JSON), but it would not work with json.loads function at all. I believe after experimentation that this was a UTF-16 related error, but it still didn't work even after specifying the encoding. Instead, I just made the PowerShell component write to a JSON file which Python can then read, and the raw JSON file may still have some use for users of the script.

If we had more time to do this project, I would have looked into producing more detailed charts, e.g. one that shows a ratio of malicious IP connections vs non malicious, or shows all IP addresses but shows clean ones as green and malicious ones as red. The data visualization function was also reusable for a basic bar chart, which would have made graphing some of this data easier to do.

References

Adam Lamont

Resources used for PowerShell

sdwheeler (2025). *Get-FileHash (Microsoft.PowerShell.Utility) - PowerShell*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash?view=powershell-7.5.

sdwheeler (2022). *Get-Process (Microsoft.PowerShell.Management) - PowerShell*. [online] Microsoft.com. Available at: https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.management/get-process?view=powershell-7.5.

(Scripting for Cyber Security Lecture Slides/Notes)

Lexie Walsh

W3 Schools (n.d.). *Python isinstance() Function*. [online] www.w3schools.com. Available at: https://www.w3schools.com/python/ref_func_isinstance.asp.

Naazneen Jatu (2021). *Python: Check if Variable is a Dictionary*. [online] Stack Abuse. Available at: https://stackabuse.com/python-check-if-variable-is-a-dictionary/#checkifvariableisadictionarywithisinstance [Accessed 12 Dec. 2025].

Stack Overflow. (n.d.). *python - How can I add new keys to a dictionary?* [online] Available at: https://stackoverflow.com/questions/1024847/how-can-i-add-new-keys-to-a-dictionary.

GeeksforGeeks (2018). *Python iter() method*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/python/python-iter-method/ [Accessed 12 Dec. 2025].

Python Software Foundation (2025). *5. Data Structures — Python 3.8.3 documentation*. [online] docs.python.org. Available at: https://docs.python.org/3/tutorial/datastructures.html.